SECURE STEGANOGRAPHY SYSTEM

Technical Documentation

Project Members
Mariam Mudassar
Anum Rafaqat
Kainat

1. Project Overview

The Secure Steganography System is a Python-based desktop application that combines two layers of data protection: AES encryption for message confidentiality and LSB (Least Significant Bit) steganography for concealing encrypted data within digital images. It also provides image-level encryption using pixel shuffling and XOR operations.

1.1 Key Features

- User authentication system with SQLite-backed login and registration
- AES-128 encryption (CBC mode) for securing secret messages
- LSB steganography to hide encrypted messages inside PNG images
- SHA-256 integrity verification to detect tampering or wrong keys
- Image encryption via row/column pixel swapping and XOR key streams
- Persistent AES key management (save/load from file)
- Metadata persistence for image encryption parameters
- Capacity checking before embedding to prevent overflow

1.2 Technology Stack

Component	Library / Technology
Language	Python 3.x
Cryptography	PyCryptodome (AES CBC)
Image Processing	Pillow (PIL), OpenCV (cv2)
Data Storage	SQLite3 (user accounts)
Visualization	Matplotlib
Encoding	Base64, SHA-256 (hashlib)
Randomness	NumPy random, Python random

2. System Architecture

The system is organized into five functional modules that work together to provide a complete secure communication pipeline:

2.1 Module Breakdown

Module	Responsibility

Login / Auth Module	SQLite-based user registration and login with SHA-256 hashed passwords
AES Crypto Module	Key generation, CBC encryption/decryption, padding/unpadding
Steganography Module	LSB encoding/decoding, capacity checking, END marker detection
Image Encryption Module	Pixel row/column swapping and XOR-based visual scrambling
Key & Meta Persistence	File-based AES key storage and JSON metadata for image decryption

2.2 Message Hide Flow

```
User Input → AES Encrypt → SHA-256 Hash → Pack (enc + hash) → LSB Encode → PNG Output
```

2.3 Message Extract Flow

```
Stego PNG → LSB Decode → Unpack (enc + hash) → AES Decrypt → Verify Hash → Display
```

2.4 Image Encryption Flow

```
Original Image → Row Swaps → Column Swaps → XOR with Key Stream → Encrypted PNG
```

3. Module Details

3.1 Authentication Module

Handles user registration and login using SQLite3. Passwords are never stored in plain text.

Function	init_db() — Creates the users table if it does not exist
Function	register_user(username, password) — Hashes password with SHA-256 and inserts into DB
Function	login_user(username, password) — Verifies hashed password against stored record
Security	Passwords stored as SHA-256 hex digest. SQLite parameterized queries prevent SQL injection.

3.2 AES Encryption Module

Implements AES-128 in CBC mode using PyCryptodome. A random 16-byte IV is generated for each encryption, prepended to the ciphertext, and encoded in Base64.

```
generate_key()          → Returns 16 random bytes (128-bit AES key)
encrypt_message(msg, key) → IV + AES_CBC_Encrypt(pad(msg)) → Base64 string
decrypt_message(enc, key) → Base64_decode → split IV → AES_CBC_Decrypt → unpad
```

Note	PKCS#7-style padding is applied manually: pad length = 16 - (len % 16), using chr(pad_len) repeated.

3.3 Steganography Module (LSB)

Uses the Least Significant Bit technique to embed data invisibly into image pixels. Each character of the payload is converted to 8 bits; each bit overwrites the LSB of one color channel (R, G, or B) of a pixel.

```
encode_image(image_path, data, output_path)  →  Hides data+"<END>" in LSBs, saves as PNG
decode_image(image_path)  →  Reads LSBs, reconstructs chars until <END> marker found
```

Capacity	Max embeddable bytes = (width × height × 3) / 8. Checked before every hide operation.

3.4 Secure Hide / Extract

Combines AES encryption, SHA-256 integrity hashing, and LSB steganography into two high-level operations:

```
secure_hide(message, image_path, output_path, key)
```

- Encrypts the message with the AES key
- Computes SHA-256 hash of the original message
- Packs them as encrypted_msg<SEP>hash
- Calls encode_image() to embed in the carrier image

```
secure_extract(image_path, key)
```

- Calls decode_image() to retrieve the packed payload
- Unpacks the encrypted message and stored hash
- Decrypts the message with the provided key
- Recomputes SHA-256 and compares — prints PASS or FAIL

3.5 Image Encryption Module

Visually scrambles an image using three reversible operations applied in sequence. Decryption reverses each step.

Step	Operation
Row Swaps	Randomly swap pairs of rows using a seeded PRNG (seed = key)
Column Swaps	Randomly swap pairs of columns (seed = key + 999)
XOR Stream	Generate a key-seeded NumPy random matrix; XOR with pixel array

Key Verify	Before decryption, SHA-256 of the entered integer key is compared to the stored hash in metadata JSON.

4. Data Flow & File Artifacts

4.1 Files Created at Runtime

File	Purpose
users.db	SQLite database holding username + hashed password
aes_key.key	Base64-encoded AES key (default filename, configurable)
<output>.png	Stego image with hidden encrypted message in LSBs
encrypted_image.png	Pixel-scrambled encrypted version of input image
decrypted_image.png	Restored original after image decryption
<name>_meta.json	Row/column swap list + key hash for image decryption

4.2 Packed Payload Format

When hiding a message, the final string embedded in the image has the structure:

```
BASE64_AES_CIPHERTEXT<SEP>SHA256_HASH_OF_PLAINTEXT
```

The <SEP> delimiter allows clean splitting during extraction. The <END> marker appended by encode_image() signals the decoder to stop reading LSBs.

5. Menu & Usage Guide

5.1 Login Menu

Option	Action
1. Login	Enter username and password to authenticate
2. Register	Create a new account (username must be unique)
3. Exit	Close the application

5.2 Main Menu

Option	Description
1. Generate AES Key	Creates a random 128-bit key and displays it as Base64
2. Save AES Key	Writes current key to a .key file (Base64 encoded)
3. Load AES Key	Reads and decodes a .key file into memory
4. Encrypt & Hide Message	Runs secure_hide() — needs key + input image + output name
5. Extract & Decrypt Message	Runs secure_extract() — needs key + stego image path
6. Encrypt Image	Scrambles a visual image using integer key; saves meta JSON
7. Decrypt Image	Reverses image scramble after verifying integer key hash
8. Exit	Exits the main menu loop

5.3 Typical Workflow — Hiding a Message

1. Login or register an account.
2. Select Option 1 to generate an AES key.
3. Select Option 2 to save the key to a file.
4. Select Option 4, enter your message, input image (.png), and desired output filename.
5. Share the output PNG with the recipient along with the .key file (via a secure channel).

6. Security Analysis

6.1 Strengths

- AES-128 CBC with random IV — secure industry-standard symmetric encryption
- SHA-256 integrity check detects wrong keys or tampered images immediately
- LSB steganography is imperceptible to the human eye for typical payloads
- Passwords stored as SHA-256 hashes — plaintext never persisted
- Parameterized SQL queries prevent injection attacks
- Image encryption key verification prevents garbage-output decryption

6.2 Limitations & Recommendations

- SHA-256 for passwords should be upgraded to bcrypt or Argon2 for production use
- The AES .key file must be transmitted securely (out-of-band); it is not protected at rest
- LSB steganography is detectable by statistical steganalysis tools
- The integer image encryption key space is limited — larger keys improve security
- No rate limiting on login attempts; brute force protection recommended for deployment

7. Installation & Requirements

7.1 Dependencies

```
pip install pycryptodome pillow opencv-python numpy matplotlib
```

7.2 Python Version

Python 3.7 or higher is required. All libraries are cross-platform (Windows, macOS, Linux).

7.3 Running the Application

```
python secure_steganography.py
```

The login menu appears on startup. Register a new user or log in to access the main menu.

7.4 Image Requirements

- Input images must be in PNG format for steganography (JPEG causes lossy compression which destroys LSB data)
- The output image is always saved as .png regardless of the extension provided
- For image encryption, any OpenCV-readable format is accepted (PNG, JPG, BMP)